

# Enterprise Knowledge Copilot — Low Level Design (LLD)

**Project:** Enterprise Knowledge Copilot  
**Total Modules:** 4 Core Python Files + 2 Evaluation Scripts  
**Lines of Code:** ~1,350+ (production logic, excluding tests)

## 1. Module Dependency Graph

The system is cleanly decoupled into four core modules with unidirectional dependencies:



**Key design principle:** `ingest.py` is an offline, one-time script. `retriever.py` only reads pre-built indices. `agent.py` orchestrates the tools. `app.py` handles only presentation. No module has circular dependencies.

## 2. Module-Level Design

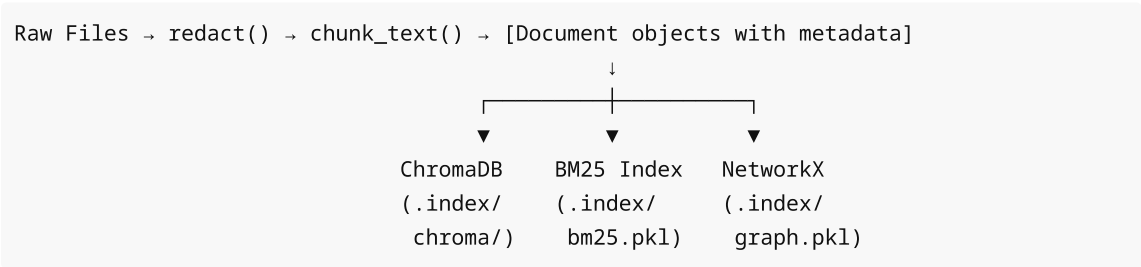
### 2.1 `src/ingest.py` — The ETL Engine (217 lines)

**Responsibility:** Reads raw CSV/TXT files, redacts PII, chunks text, and builds three independent search indices.

**Key Functions:**

| Function                                             | Purpose                                         | Implementation Detail                                                                                                                                  |
|------------------------------------------------------|-------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>redact(text)</code>                            | Strips PII from raw text                        | Regex for emails/phones → spaCy NER for PERSON entities → Sorts by string length (longest first) to prevent partial leaks                              |
| <code>chunk_text(text, size=800, overlap=150)</code> | Splits text into overlapping windows            | Rolling window ensures no sentence is split without appearing in at least one complete chunk                                                           |
| <code>load_text_files(data_dir)</code>               | Parses <code>.txt</code> handbook/runbook files | Tags each chunk with <code>source</code> , <code>chunk_index</code> , <code>type=handbook</code> , and detected system name                            |
| <code>load_csv_files(data_dir)</code>                | Parses <code>.csv</code> ticket/org files       | Extracts structured metadata: <code>priority</code> , <code>status</code> , <code>error_code</code> , <code>ticket_id</code> , <code>created_at</code> |
| <code>build_graph_from_csv(data_dir)</code>          | Constructs the NetworkX DiGraph                 | Loads explicit triples from org chart (entity→relationship→target) + discovers co-mentioned entities via NER                                           |
| <code>build_index()</code>                           | Master orchestrator                             | Calls all loaders, builds ChromaDB (vector), BM25 (keyword), and NetworkX (graph) indices. Records ingest timestamp.                                   |

**Data Flow:**



### 2.2 `src/retriever.py` — The Three-Headed Search Engine (231 lines)

**Responsibility:** Executes queries against all three indices, fuses results with RRF, and reranks with a Cross-Encoder.

**Key Functions:**

| Function | Purpose | Implementation Detail |
|----------|---------|-----------------------|
|----------|---------|-----------------------|

|                                                           |                          |                                                                                                                                             |
|-----------------------------------------------------------|--------------------------|---------------------------------------------------------------------------------------------------------------------------------------------|
| <code>expand_query(query)</code>                          | Abbreviation resolution  | Maps "vpn" → "NEXAVPN", "ci" → "BUILDDPIPE-CI", "password" → "credential password reset" via a 24-entry lookup table                        |
| <code>search_semantic(query, k, filter_dict)</code>       | ChromaDB vector search   | Uses <code>bge-small-en-v1.5</code> embeddings (33MB, CPU-friendly). Supports metadata filtering via ChromaDB's <code>\$and</code> operator |
| <code>search_keyword(query, k, source_filter)</code>      | BM25 sparse retrieval    | Tokenizes query, scores against pre-built BM25 corpus, filters by source file                                                               |
| <code>search_graph(entities)</code>                       | NetworkX graph traversal | Finds matching nodes, traverses both outgoing and incoming edges, deduplicates, returns up to 20 triples                                    |
| <code>_rrf_fuse(query, sem_results, kw_results, k)</code> | Reciprocal Rank Fusion   | $score = \sum 1/(60 + rank)$ — normalizes across different scoring scales. Feeds top 2k candidates to Cross-Encoder                         |
| <code>_rerank(query, docs, k)</code>                      | Cross-Encoder reranking  | Loads <code>ms-marco-MiniLM-L-6-v2</code> (lazy-loaded). Pushes (query, doc) pairs through transformer together. Returns top-k by relevance |
| <code>compute_semantic_similarity(query, answer)</code>   | Cosine similarity        | Embeds both texts, computes vector dot product. Used for deduplication and evaluation metrics                                               |
| <code>compute_context_relevance(query, documents)</code>  | Average chunk similarity | Embeds query + each retrieved chunk, averages cosine similarities. Measures retrieval quality                                               |

#### Retrieval Pipeline (per query):

```

Query → expand_query() → [search_semantic() + search_keyword()]
      ↓
      _rrf_fuse() → Top 10 merged
      ↓
      _rerank() → Top 5 final (Cross-Encoder)

```

### 2.3 `src/agent.py` — The Agentic Brain (600 lines)

**Responsibility:** Defines the LangGraph state machine, implements 6 tools, handles routing, escalation, metric computation, and audit logging.

**State Schema** (TypedDict):

| Field                 | Type                   | Purpose                                               |
|-----------------------|------------------------|-------------------------------------------------------|
| <code>question</code> | <code>str</code>       | The user's natural language query                     |
| <code>entities</code> | <code>list[str]</code> | Extracted system names, error codes, and NER entities |

|                     |            |                                                                                  |
|---------------------|------------|----------------------------------------------------------------------------------|
| route               | str        | Classified tool route: docs / tickets / summarize / filtered / create / multihop |
| documents           | list       | Retrieved LangChain Document objects                                             |
| graph_context       | str        | Knowledge Graph relationships as formatted text                                  |
| retrieval_score     | float      | Average RRF score from retrieval                                                 |
| answer              | str        | The LLM's generated response                                                     |
| citations           | list[dict] | Source documents with snippets for provenance                                    |
| tool_used           | str        | Human-readable label of the selected tool                                        |
| faithfulness        | float      | Token overlap between answer and source context                                  |
| semantic_similarity | float      | Cosine similarity between query and answer embeddings                            |
| context_relevance   | float      | Average cosine similarity between query and retrieved chunks                     |
| confidence          | float      | Weighted composite score displayed to user                                       |
| reasoning_trace     | list[str]  | Step-by-step log of the agent's internal decisions                               |
| history             | list[dict] | Previous conversation turns for multi-turn context                               |
| role                | str        | User's access tier (Employee / Manager / IT Admin)                               |

#### The 6 Tools:

1. **tool\_search\_docs** : Queries handbook + runbook via hybrid search. Also traverses the Knowledge Graph for any detected entities. Returns up to 5 reranked documents.
2. **tool\_search\_tickets** : Searches historical ticket data. Enriches results with graph relationships for system-owner mapping.
3. **tool\_filtered\_tickets** : Performs metadata-aware searches. Extracts priority level (P1-P4) and system name from the query, applies them as ChromaDB filter conditions.
4. **tool\_summarize** : Broad search across ALL data sources (k=8) for overview-style questions.
5. **tool\_multihop** : Decomposes complex questions into three sub-tasks (Hop 1: handbook definitions → Hop 2: historical tickets → Hop 3: graph relationships), then synthesizes all results.
6. **tool\_create\_ticket** : Before creating a ticket, searches for similar open tickets. If cosine similarity > 0.75, blocks creation and shows the duplicate. Otherwise, writes a new row to the CSV with auto-generated ticket ID, priority, and error code.

## 2.4 src/app.py — The Presentation Layer (311 lines)

**Responsibility:** Streamlit session state management, UI rendering, and user interaction handling.

#### Key Components:

| Component               | Implementation                                                                      |
|-------------------------|-------------------------------------------------------------------------------------|
| <b>Custom CSS Theme</b> | Dark gradient background, glassmorphism cards, Inter font, color-coded metric pills |

|                          |                                                                                                 |
|--------------------------|-------------------------------------------------------------------------------------------------|
| <b>Sidebar Dashboard</b> | Reads bm25.pkl, graph.pkl, and ingest_meta.pkl to display live system health stats              |
| <b>Role Selector</b>     | Dropdown for Employee/Manager/IT Admin — controls data access tier                              |
| <b>Onboarding Mode</b>   | 10 curated questions in expandable cards with on-demand answer generation                       |
| <b>Chat Rendering</b>    | Displays tool badges, confidence badges, metric pills, citation expanders, and reasoning traces |
| <b>Feedback System</b>   | Thumbs up/down buttons per response → logged to feedback.jsonl                                  |
| <b>Suggestion Chips</b>  | 6 pre-built example queries shown when chat history is empty                                    |

### 3. Error Handling & Resilience

| Scenario                    | Handling Strategy                                                                     |
|-----------------------------|---------------------------------------------------------------------------------------|
| Cross-Encoder fails to load | Graceful fallback: <code>_cross_encoder = False</code> , system uses RRF-only ranking |
| Zero retrieval results      | Agent bypasses LLM entirely → Smart Escalation with Knowledge Graph contacts          |
| Malformed CSV rows          | Robust DictReader parsing with <code>.strip().strip('\"')'</code> on all fields       |
| Duplicate ticket creation   | Cosine similarity check (>0.75 threshold) blocks duplicates before CSV write          |
| spaCy model not installed   | Script fails fast with a clear import error, not a silent degradation                 |
| Ollama server offline       | LangChain raises a connection error surfaced in the Streamlit UI                      |

### 4. File-Level Dependency Map

```
requirements.txt
├─ langchain-community      # ChromaDB integration
├─ langchain-huggingface    # HuggingFace embeddings
├─ langchain-ollama         # Local LLM connection
├─ langgraph               # State machine framework
├─ chromadb                # Vector database
├─ rank-bm25               # BM25 keyword search
├─ networkx                # Knowledge graph
├─ sentence-transformers    # Cross-encoder reranker
├─ spacy                   # NER for PII redaction
├─ streamlit               # Frontend UI
└─ numpy                   # Vector math
```